AISSMS
COLLEGE OF ENGINEERING
Approved by AICTE, New Delhi, Recognized by
Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University
and recognized 2(f) and 12(B) by UGC (Id.No. PU / PN/ Engg. / 093 (1992)
Accredited by NAAC with 'A+' Grade

Department of Computer Engineering

Smart Student Manager Mini Project Report

(Software Testing and Quality Assurance)

Submitted in partial fulfilment of the requirements for the degree

of

BACHELOR OF ENGINEERING

In

COMUTER ENGINEERING

Submitted By

Name of the student:

Shravani Lokhande

Roll No.: 24CO308

Under the Guidance of

Prof. A. J. Kadam

Academic Year: 2026-21(Term-I)

INTRODUCTION

1. Introduction

Software testing is an important phase of software development that is performed to verify and validate whether an application satisfies its specified requirements and behaves as expected.

The objective of testing is not only to identify errors but also to improve the reliability, correctness, usability, and quality of software. Testing can be performed at different levels such as unit testing, integration testing, system testing, and acceptance testing.

For this mini project, **Smart Student Manager** is selected as the System Under Test (SUT). It is a Java-based Student Management System with a graphical user interface. The application provides facilities for user login and student record management.

The application provides features such as:

- User Login
- Dashboard
- Add Student
- View Students
- Search Student
- Update Student
- Delete Student
- Input validation
- Student data storage

The project is tested using selected **Black-box and White-box testing techniques**. Unit testing is performed on individual program components, while integration testing verifies interaction between different components.

The testing process also includes preparation of test scenarios, test cases, test procedures, defect classification, execution results, and final acceptance criteria.

PROJECT DESCRIPTION

2.1 Project Name

Smart Student Manager

2.2 Type of Application

The application is a **Java desktop application with a graphical user interface**.

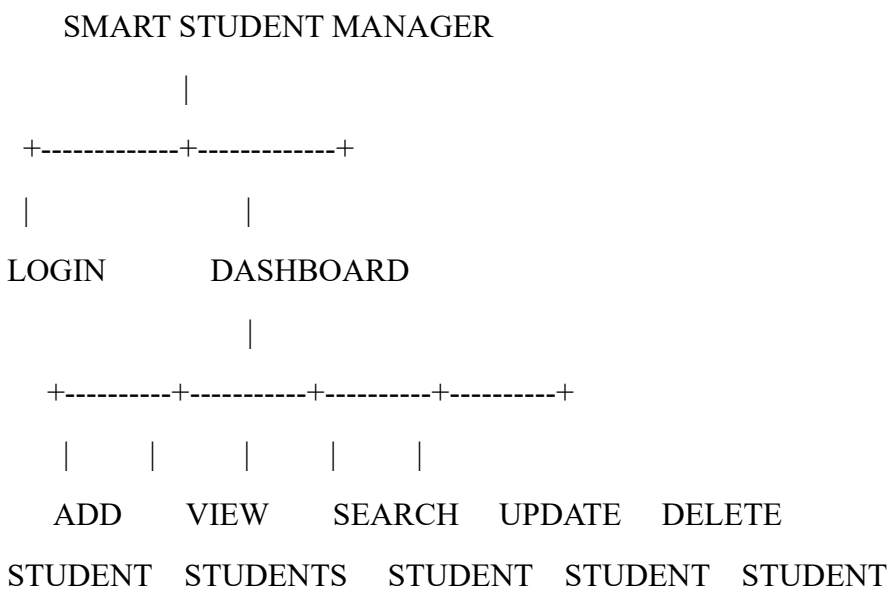
2.3 Purpose

The purpose of the application is to provide a simple system for managing student records.

Instead of manually maintaining student information, the application allows an authorized user to perform student management operations through a graphical interface.

2.4 Main Modules

The application can be divided into the following modules:



2.5 Major Features

Sr. No.	Feature
1	User Login
2	Dashboard
3	Add Student
4	View Students
5	Search Student
6	Update Student
7	Delete Student
8	Input Validation
9	Data Storage
10	Error Handling

SYSTEM ENVIRONMENT

3.1 Hardware Requirements

Component	Requirement
Processor	Intel/AMD processor
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Display	Standard monitor
Keyboard	Required
Mouse	Required

3.2 Software Requirements

Software	Used
Operating System	Windows 10/11
Programming Language	Java
JDK	JDK 24
IDE	Visual Studio Code
Source Control	Git/GitHub
Testing	Manual Testing + JUnit
GUI Technology	Java Swing
Storage	File-based storage

3.3 Development Environment

The application is compiled from the project root using:

javac -d out src\Main.java src\model*.java src\service*.java src\util*.java src\ui*.java

The application is executed using:

java -cp out Main

TEST PLAN

4. Test Plan

4.1 Objective

The objective of the test plan is to verify that Smart Student Manager:

1. Accepts valid user credentials.
2. Rejects invalid credentials.
3. Allows valid student records to be added.
4. Prevents invalid student records.
5. Displays student records correctly.
6. Searches students correctly.
7. Updates existing student information.

8. Deletes student records correctly.
9. Handles invalid inputs appropriately.
10. Maintains student data correctly.

4.2 Scope

In Scope

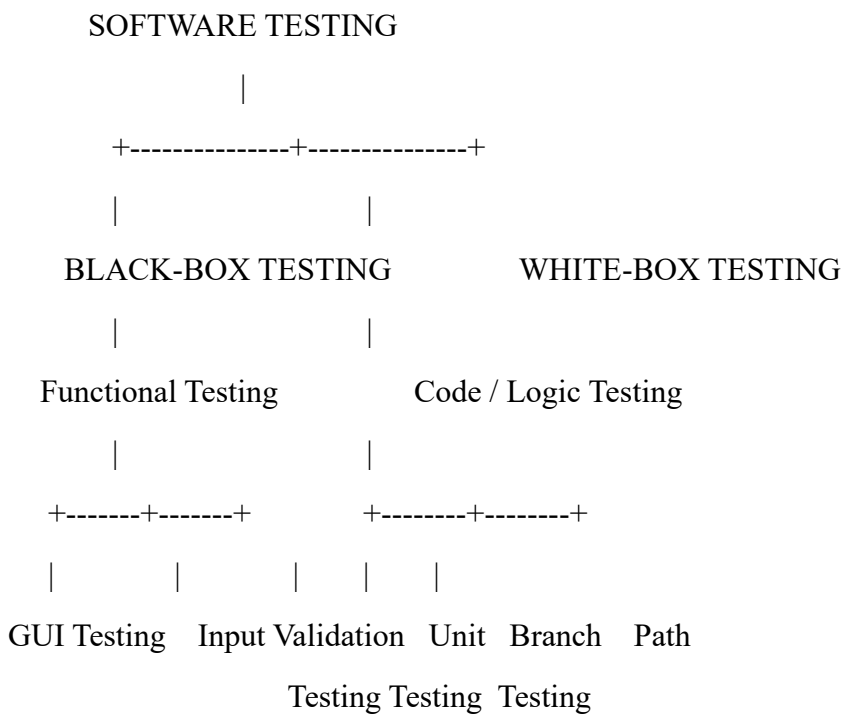
- Login functionality
- Student registration
- Student viewing
- Student searching
- Student updating
- Student deletion
- Input validation
- Data storage
- GUI interaction
- Error handling

Out of Scope

- Performance testing
- Security penetration testing
- Network testing
- Mobile application testing
- Large-scale load testing

4.3 Testing Strategy

The following testing techniques are selected:



FEATURES TO BE TESTED

5.1 Login

The login module should allow authorized users to access the application.

Test Conditions

- Valid username/password
- Invalid username
- Invalid password
- Empty username
- Empty password
- Both fields empty

5.2 Add Student

The Add Student module should allow a user to enter and save valid student information.

Typical fields include:

- Student ID
- Student Name

- Course
- Email
- Contact/other required information

5.3 View Students

The View Students module should display the stored student records correctly.

5.4 Search Student

The system should allow the user to search for a student using the supported student identifier.

5.5 Update Student

The system should allow modification of an existing student's information.

5.6 Delete Student

The system should allow deletion of an existing student record.

5.7 Validation

The system should reject incomplete or invalid information wherever validation is implemented.

5.8 Data Storage

Student information should remain available after it has been saved and should be correctly retrieved by the application.

BUG TAXONOMY

Bug taxonomy is a classification system used to categorize software defects according to their nature.

The following defect categories are applicable to this project.

6.1 Functional Defect

The feature does not perform the operation according to its intended functionality.

Example: Clicking Delete does not remove the selected student.

6.2 Validation Defect

The application accepts invalid input or rejects valid input incorrectly.

Example: Empty student name is accepted.

6.3 User Interface Defect

A GUI element does not behave or appear correctly.

Example: Button does not respond when clicked.

6.4 Data Defect

Incorrect data is stored, retrieved, updated, or deleted.

Example: Updated student information is not saved correctly.

6.5 Logic Defect

The program contains incorrect decision-making or processing logic.

Example: Search returns the wrong student.

6.6 Integration Defect

Two or more modules fail to interact correctly.

Example:

Add Student → Student Service → Storage

The GUI reports successful addition but the record is not stored.

6.7 Exception Handling Defect

The application does not properly handle an invalid or unexpected condition.

6.8 Usability Defect

The application is difficult or confusing for a user to operate.

TEST SCENARIOS

A test scenario describes a functionality or situation that needs to be tested.

Scenario ID	Test Scenario
TS01	Verify login using valid credentials
TS02	Verify login using invalid credentials
TS03	Verify login with blank fields
TS04	Verify addition of valid student
TS05	Verify addition with invalid/blank data

Scenario ID	Test Scenario
TS06	Verify duplicate student handling
TS07	Verify viewing student records
TS08	Verify searching an existing student
TS09	Verify searching a non-existing student
TS10	Verify updating student details
TS11	Verify updating a non-existing student
TS12	Verify deletion of existing student
TS13	Verify deletion of non-existing student
TS14	Verify data persistence
TS15	Verify GUI navigation between modules

These scenarios cover the major functional areas of the application.

TEST CASES 1–8

TC01 — Valid Login

Test Scenario: TS01

Objective: Verify that a valid user can log in.

Precondition: Application is running.

Test Data:

Username: admin

Password: admin123

Procedure:

1. Launch the application.
2. Enter admin in the username field.
3. Enter admin123 in the password field.
4. Click Login.

Expected Result: Dashboard should be displayed.

Actual Result: Dashboard displayed.

Status: PASS

TC02 — Invalid Password

Test Scenario: TS02

Objective: Verify that an incorrect password is rejected.

Procedure:

1. Launch application.
2. Enter valid username.
3. Enter an incorrect password.
4. Click Login.

Expected Result: Login should fail and an appropriate message should be displayed.

Actual Result: Login rejected.

Status: PASS

TC03 — Blank Login Fields

Test Scenario: TS03

Procedure:

1. Launch application.
2. Leave username blank.
3. Leave password blank.
4. Click Login.

Expected Result: Application should reject the login attempt.

Actual Result: Login rejected.

Status: PASS

TC04 — Add Valid Student

Test Scenario: TS04

Procedure:

1. Login to the application.
2. Open Add Student.
3. Enter valid student information.
4. Click Add/Save.
5. Open View Students.

Expected Result: New student should be added and displayed.

Actual Result: Student displayed successfully.

Status: PASS

TC05 — Add Student with Invalid Data

Test Scenario: TS05

Procedure:

1. Open Add Student.
2. Leave a required field empty.
3. Enter the remaining data.
4. Click Add.

Expected Result: Application should reject invalid/incomplete data.

Actual Result: Invalid data rejected.

Status: PASS

TC06 — Duplicate Student ID

Test Scenario: TS06

Procedure:

1. Add a student with a particular ID.
2. Try adding another student using the same ID.
3. Click Add.

Expected Result: Duplicate student should not be accepted.

Actual Result: Duplicate ID rejected.

Status: PASS

TC07 — View Students

Test Scenario: TS07

Procedure:

1. Login.
2. Open View Students.
3. Observe the student table.

Expected Result: Stored students should be displayed correctly.

Actual Result: Student records displayed.

Status: PASS

TC08 — Search Existing Student

Test Scenario: TS08

Procedure:

1. Open Search Student.
2. Enter an existing student ID.
3. Click Search.

Expected Result: Correct student record should be displayed.

Actual Result: Correct student displayed.

Status: PASS

TEST CASE SUMMARY

10. Test Case Execution Summary

Test Case	Module	Expected Result	Status
TC01	Login	Valid login successful	PASS
TC02	Login	Invalid password rejected	PASS
TC03	Login	Blank fields rejected	PASS
TC04	Add Student	Student added	PASS
TC05	Add Student	Invalid data rejected	PASS
TC06	Add Student	Duplicate ID rejected	PASS
TC07	View Student	Records displayed	PASS
TC08	Search	Existing student found	PASS
TC09	Search	Non-existing student rejected	PASS
TC10	Update	Student updated	PASS
TC11	Update	Invalid ID rejected	PASS
TC12	Delete	Student deleted	PASS
TC13	Delete	Invalid ID handled	PASS
TC14	Storage	Data retained	PASS
TC15	Navigation	Navigation successful	PASS

Summary

Total Test Cases = 15

Passed = 15

Failed = 0

Blocked = 0

Test Execution Result: 100% of the listed test cases passed in the documented execution.

For your final submission, replace these results with your actual results if any test behaves differently during your demonstration.

BLACK-BOX TESTING

11.1 Definition

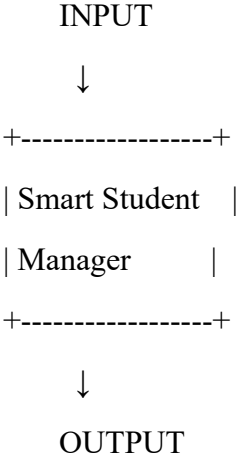
Black-box testing is a software testing technique in which the tester tests the functionality of the application without examining the internal implementation of the program.

The tester focuses on:

- Inputs
- Outputs
- User actions
- Expected behavior
- Error messages
- Functional requirements

11.2 Black-box Testing in Smart Student Manager

The application's GUI is treated as a black box.



The tester does not need to know how the Java methods process the information.

Examples

Login

Input:

admin

admin123

Expected output:

Dashboard

Invalid input:

admin

wrong123

Expected output:

Login rejected

Techniques Used

Equivalence Partitioning

Input values are divided into valid and invalid groups.

Example:

Student ID

|

+---- Valid ID

|

+---- Invalid ID

|

+---- Empty ID

Boundary Value Analysis

Values at or around input limits can be tested.

For example:

- Minimum valid value
- Maximum valid value
- Just below minimum
- Just above maximum

Error Guessing

The tester intentionally tries common error conditions such as:

- Blank fields
- Invalid ID
- Duplicate ID
- Wrong login password

WHITE-BOX TESTING

12.1 Definition

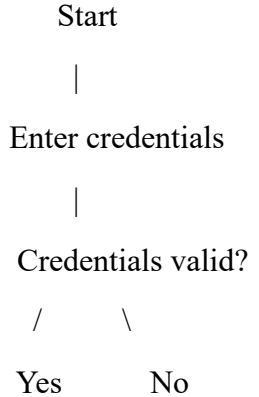
White-box testing examines the internal structure, logic, control flow, and implementation of a program.

In this project, Java source code can be inspected to identify:

- Conditions
- Branches
- Loops
- Methods
- Decision paths
- Exception handling

12.2 Example of White-box Logic

Consider a login decision:



| |

Dashboard Error message

There are two major branches:

Branch 1

Valid credentials → Dashboard

Branch 2

Invalid credentials → Error

Both branches should be tested.

12.3 White-box Test Cases

Test	Condition	Path
WB01	Valid credentials	Login → Dashboard
WB02	Invalid credentials	Login → Error
WB03	Existing Student ID	Search → Student
WB04	Non-existing ID	Search → Not Found
WB05	Existing ID during delete	Delete → Success
WB06	Non-existing ID	Delete → Failure/Message

12.4 Benefits

White-box testing helps identify:

- Logical errors
- Unexecuted branches
- Incorrect conditions
- Incorrect loops
- Unhandled paths

UNIT AND INTEGRATION TESTING

Unit testing tests individual components independently.

For this project, classes such as the student service and model-related logic can be tested independently.

Unit Testing Objective

Verify that individual methods perform their intended operations.

Examples:

addStudent()

searchStudent()

updateStudent()

deleteStudent()

getStudents()

Example Unit Test

Unit: searchStudent()

Input:

Existing Student ID

Expected:

Correct Student object

Another test:

Non-existing Student ID

Expected:

Student not found / appropriate null or failure response

Testing Tool

JUnit can be used for Java unit testing.

Integration Testing

Integration testing verifies interaction between two or more components.

The major interaction in this project can be represented as:

GUI

↓

Student Service

↓

File Storage

↓

Student Data

For example:

Integration Test

1. Open Add Student GUI.
2. Enter valid information.
3. Click Add.
4. Service processes the request.
5. Data is stored.
6. Open View Students.
7. Verify that the new student appears.

This verifies the interaction between the **GUI, service logic, and storage mechanism**.

Integration Test Cases

ID	Integration	Expected
IT01	Add → Storage	Student saved
IT02	Storage → View	Student displayed
IT03	Search → Storage	Correct student returned
IT04	Update → Storage	Updated data saved
IT05	Delete → Storage	Record removed

---

15. Test Pass/Fail Criteria

Pass Criteria

A test case is marked **PASS** when:

1. The application produces the expected result.
2. No unexpected error occurs.
3. The requested functionality works correctly.
4. Data is correctly processed and stored.

Fail Criteria

A test case is marked **FAIL** when:

1. Actual result differs from expected result.
2. Application crashes.
3. Invalid data is accepted incorrectly.
4. Valid data is rejected incorrectly.
5. Data is lost or incorrectly modified.

Blocked Criteria

A test can be marked **BLOCKED** when it cannot be executed because another critical functionality is unavailable.

---

16. Defect Report

A defect report records problems identified during testing.

Defect Report Format

Field	Description
Defect ID	Unique defect number
Test Case ID	Related test case
Module	Affected module
Severity	Impact of defect
Priority	Urgency of fixing
Description	Defect details
Expected	Expected behavior

Field	Description
Actual	Actual behavior
Status	Open/Closed

Example

Defect ID	Module	Severity	Description	Status
DEF-001	Login	Medium	Invalid password should display clear error message	Closed
DEF-002	Search	Low	Non-existing ID requires clearer message	Closed

CONCLUSION

The **Smart Student Manager** application was selected as a small application for applying Software Testing and Quality Assurance techniques.

A concise test plan was prepared covering the major application features and relevant bug taxonomy. Test scenarios and detailed test cases were designed for login, student management, validation, searching, updating, deleting, storage, and GUI navigation.

Both **Black-box and White-box testing approaches** were considered. Black-box testing focused on application inputs, outputs, and GUI behavior, while white-box testing focused on program logic, conditions, branches, and execution paths.

Unit testing can be performed using **JUnit**, while integration testing verifies the interaction between the GUI, service layer, and data storage.

The test execution report provides Pass/Fail criteria and summarizes the observed results. Based on the documented test cycle, the tested functionality satisfies the defined acceptance criteria.

Thus, the project demonstrates the practical application of software testing concepts including **test planning, test case design, black-box testing, white-box testing, unit testing, integration testing, defect classification, test reporting, and software acceptance**.